TANKISO MASOEBE

PROGRAMMING WITH C++WEEK12

1.Structure Design Challenge Submission

Scenario Chosen: Bank Account System

Struct Definition (C Language)

```
struct BankAccount {
    char accountHolderName[100];
    int accountNumber;
    float balance;
    char accountType[20];
    bool isActive;
};
```

Explanation of Members

- **accountHolderName (char[100])**
  Stores the name of the account holder. A character array is used to hold text.
- **accountNumber (int)**
  A unique number assigned to each bank account.
- **balance (float)**
  Represents the amount of money in the account, including decimals.
- **accountType (char[20])**
  Stores the type of account (e.g., savings, current).
- **isActive (bool)**
  Indicates whether the account is active (true) or closed (false).

Justification of Design

The struct includes all essential information needed to represent a bank account. Appropriate data types were chosen based on the type of data being stored:

- Text fields use character arrays.
- Numerical values use integers or floating-point numbers.
- The account status is represented using a boolean for simplicity and clarity.

2. Here's a **complete, ready-to-submit answer** for your **Code Correction task**:

2.Code Correction: Structures in C

Incorrect Code with Errors

```
#include <stdio.h>

struct BankAccount {
    char name[50]
    int accountNumber;
    float balance;
}

void deposit(struct BankAccount acc, float amount) {
    acc.balance = acc.balance + amount;
}

int main() {
    struct BankAccount user1;

    user1->accountNumber = 12345;
    user1.balance = 1000;

    deposit(user1, 500);

    printf("Balance: %f", user1.balance);

    return 0;
}
```

Errors Identified

1. Missing semicolon (;) after `char name[50]`
2. Missing semicolon (;) after struct definition
3. Incorrect member access using `->` instead of `.`
4. Structure passed **by value** instead of **by reference**, so changes don't persist

---

Corrected Code

```
#include <stdio.h>

struct BankAccount {
    char name[50];
```

```
    int accountNumber;
    float balance;
};

void deposit(struct BankAccount *acc, float amount) {
    acc->balance = acc->balance + amount;
}

int main() {
    struct BankAccount user1;

    user1.accountNumber = 12345;
    user1.balance = 1000;

    deposit(&user1, 500);

    printf("Balance: %f", user1.balance);

    return 0;
}
```

Explanation of Corrections

- Added missing semicolons to fix syntax errors.
- Replaced `->` with `.` when accessing struct members directly.
- Changed the function parameter to a **pointer (`*acc`)** so the original structure can be modified.
- Used `&user1` to pass the address of the structure to the function.
- Used `->` inside the function since a pointer is being used.

Conclusion

The corrected code now compiles successfully and updates the account balance correctly. Using pointers ensures that modifications inside functions affect the original structure.